

Double-layer Sarrus and pairwise X-cell constraints in WaveVis

Alan N. Pham

AO Labs, WaveVis simulator, linkage constraint paper

This paper explains the double-layer Sarrus / X-cell branch of WaveVis. The point is not to draw a prettier wave. The point is to decide which local mechanism constraints can stay physically meaningful while the sheet tries to form a localized overhang. The paper records the accepted pairwise X-cell grammar, the rejected all-four-equal cell grammar, the connector occupancy rule, the equations behind the current solver, and the exact boundary between a mechanism test and a reference-only surface.

1 Introduction

Sarrus-like linkages matter here because they convert constrained bars and pivots into a guided out-of-plane motion. A classical Sarrus mechanism is an overconstrained spatial linkage: it can move even though a naive mobility count would suggest it should lock. That history is useful, but WaveVis is not currently a complete Sarrus mechanism synthesis package. It is a design workbench for testing whether a sheet of repeated local cells can keep a real linkage grammar while it approaches a desired deformation.

The relevant literature gives two warnings. First, overconstrained mechanisms must be checked by closure equations, not by visual plausibility alone (1–3). Second, architected sheets, origami tubes, auxetic patterns, and mechanical metamaterials can look continuous while their local constraints do most of the real work (4–8). WaveVis failed in earlier iterations whenever it drew a surface and let the user infer the mechanism. The current rule is the opposite: the mechanism is the evidence.

2 Implementation Source

24 The active mechanism source is `src/xCellMechanism.ts`. The mechanism consumes a
 25 `LatticeModel` from `src/latticeGeometry.ts`. It then constructs one visible X-cell
 26 frame per lattice node and one shared connector per adjacent center pair. Rendering hap-
 27 pens in `src/LatticeViewer3D.tsx`. The verification script for this branch is `scripts/
 28 check-geometry-invariants.cjs`.

Term	Meaning in the active model
Cell center	One point associated with one lattice node. It may be smoothed for display/mechanism readability, but it remains the center of one visible cell body.
X-cell	One cell body with four straight legs: NE, NW, SE, and SW.
Connector	One physical node shared by exactly two adjacent cells.
Opposite pair	NE/SW or NW/SE pair inside one cell. Each pair is collinear and equal-length about the center.
Rejected all-four rule	The older rule that tried to make all four legs equal while also making all adjacent connectors shared across a deformed overhang. It failed the sampled overhang closure test.

Table 1. Mechanism vocabulary. These are not decorative labels; they define the acceptance tests for the public render.

29 3 Why the All-Four-Equal Cell Failed

30 The most tempting X-cell rule is:

$$\|\mathbf{p}_{NE} - \mathbf{c}\| = \|\mathbf{p}_{NW} - \mathbf{c}\| = \|\mathbf{p}_{SE} - \mathbf{c}\| = \|\mathbf{p}_{SW} - \mathbf{c}\|,$$

31 with both diagonals crossing at $\mathbf{c}$. For one isolated cell this is easy. The problem appears when
 32 neighboring cells must share connectors. If two adjacent cells share a connector $\mathbf{q}_{ij}$, then that connector
 33 must satisfy both cells' leg-length and centerline equations. Across a curved or overhanging sheet, the
 34 same point is pulled by incompatible local equal-length constraints.

35 The old proof record encoded this as a linear closure check over the sampled overhang. The active
 36 source record states a 44×44 sampled overhang test with 1764 equations and 352 variables. The
 37 all-four-equal branch produced residual norm 13.837 and RMS residual 0.329. The important point is
 38 not that those numbers are universal; the point is that the current target field makes the all-four-equal
 39 closure fail for the tested discretization. A candidate that hides that residual with a smooth surface is

not better. It is less honest.

40

4 Accepted Pairwise X-Cell Constraint

41

The accepted branch relaxes one thing and preserves another. It does not require all four legs of a cell to share one length. It does require each opposite pair to be straight and equal about the cell center:

42

43

$$\mathbf{p}_{NE} - \mathbf{c} = -(\mathbf{p}_{SW} - \mathbf{c}),$$

44

$$\mathbf{p}_{NW} - \mathbf{c} = -(\mathbf{p}_{SE} - \mathbf{c}).$$

Equivalently,

45

$$\frac{\mathbf{p}_{NE} + \mathbf{p}_{SW}}{2} = \mathbf{c}, \quad \frac{\mathbf{p}_{NW} + \mathbf{p}_{SE}}{2} = \mathbf{c}.$$

This keeps one visible black square cell body and four straight legs. It avoids pretending that a more restrictive all-four-equal cell is still active when it is not.

46

47

4.1 Adjacent-pair connectors

For a row of centers $\mathbf{c}_0, \dots, \mathbf{c}_{n-1}$, a connector exists between every adjacent pair. The connector count is $m = n - 1$. The current source solves one chain at a time. For a chain, it constructs alternating signs $s_k = (-1)^k$ and offsets $\mathbf{o}_k$:

$$s_0 = 1, \quad \mathbf{o}_0 = \mathbf{0},$$

$$s_k = -s_{k-1}, \quad \mathbf{o}_k = 2\mathbf{c}_k - \mathbf{o}_{k-1}.$$

It then chooses one free vector $\mathbf{f}$ so connectors are as close as possible to center-pair midpoints:

$$\mathbf{f} = \frac{1}{m} \sum_{k=0}^{m-1} s_k \left(\frac{\mathbf{c}_k + \mathbf{c}_{k+1}}{2} - \mathbf{o}_k \right).$$

The connector is

$$\mathbf{q}_k = \mathbf{o}_k + s_k \mathbf{f}.$$

For a two-center chain, this reduces to the midpoint

$$\mathbf{q}_0 = \frac{\mathbf{c}_0 + \mathbf{c}_1}{2}.$$

The chain solve is repeated for east-west and north-south adjacent families. The east-west family maps the connector to NE of the left cell and SW of the right cell. The north-south family maps the connector to SE of the upper cell and NW of the lower cell.

5 Connector Occupancy

The connector rule is physical, not semantic. It is not enough for a data structure to say that every connector ID has two uses. Two different connector IDs can occupy the same visible point, which creates a four-cell junction while the code still claims each ID has two uses. WaveVis therefore groups connector positions by rounded physical coordinate and counts endpoint uses per physical point:

$$U(\mathbf{q}) = \sum_{\text{ids } k: \mathbf{q}_k \approx \mathbf{q}} u_k.$$

The gate is

$$U(\mathbf{q}) \leq 2.$$

An over-occupied physical connector is a mechanism failure even if the visible surface looks smoother.

6 Cell-Body Visibility

The public render must show:

- one cell body per lattice node;
- four straight legs per interior cell;
- shared spherical connector nodes;
- no hidden filler bridge;
- no projected fake endpoint that only exists to make a line look attached;
- no surface-only silhouette standing in for the mechanism.

The mechanism can be simplified for readability, but it cannot be replaced by a skin while the UI still claims the mechanism is being shown.

7 Metrics

For each cell frame, WaveVis measures the following residuals.

7.1 Opposite-pair length spread

For pair a, b across center $\mathbf{c}$,

$$\Delta L = ||\mathbf{p}_a - \mathbf{c}|| - ||\mathbf{p}_b - \mathbf{c}||.$$

The maximum across all checked pairs is reported as `maxOppositePairLengthSpread`.

7.2 Opposite-pair collinearity

The collinearity error is

$$\theta_{\text{col}} = \cos^{-1} \left(\frac{(\mathbf{p}_a - \mathbf{c}) \cdot (\mathbf{c} - \mathbf{p}_b)}{||\mathbf{p}_a - \mathbf{c}|| ||\mathbf{c} - \mathbf{p}_b||} \right).$$

If the pair is perfectly straight through the center, $\theta_{\text{col}} = 0$.

7.3 Center residual

The center residual is

$$r_c = \left\| \frac{\mathbf{p}_a + \mathbf{p}_b}{2} - \mathbf{c} \right\|.$$

This catches cases where two legs are parallel-looking but do not actually bisect at the cell center.

7.4 Connector endpoint gap

For connector $\mathbf{q}$ and its attached endpoint $\mathbf{p}$,

$$r_q = \|\mathbf{q} - \mathbf{p}\|.$$

The public render must not draw a leg endpoint away from the shared connector and then cover the gap with a fake coupler.

7.5 Connector line residual and parameter

For connector $\mathbf{q}$ between centers $\mathbf{c}_0$ and $\mathbf{c}_1$, define

$$t = \frac{(\mathbf{q} - \mathbf{c}_0) \cdot (\mathbf{c}_1 - \mathbf{c}_0)}{\|\mathbf{c}_1 - \mathbf{c}_0\|^2}.$$

The nearest point on the center segment is

$$\mathbf{q}_{\parallel} = \mathbf{c}_0 + t(\mathbf{c}_1 - \mathbf{c}_0).$$

The line residual is $\|\mathbf{q} - \mathbf{q}_{\parallel}\|$. Overshoot is $\max(0, -t, t - 1)$. A connector that moves far off the center-pair segment is a tradeoff and must be reported.

8 Accepted Figures and Failure Figures

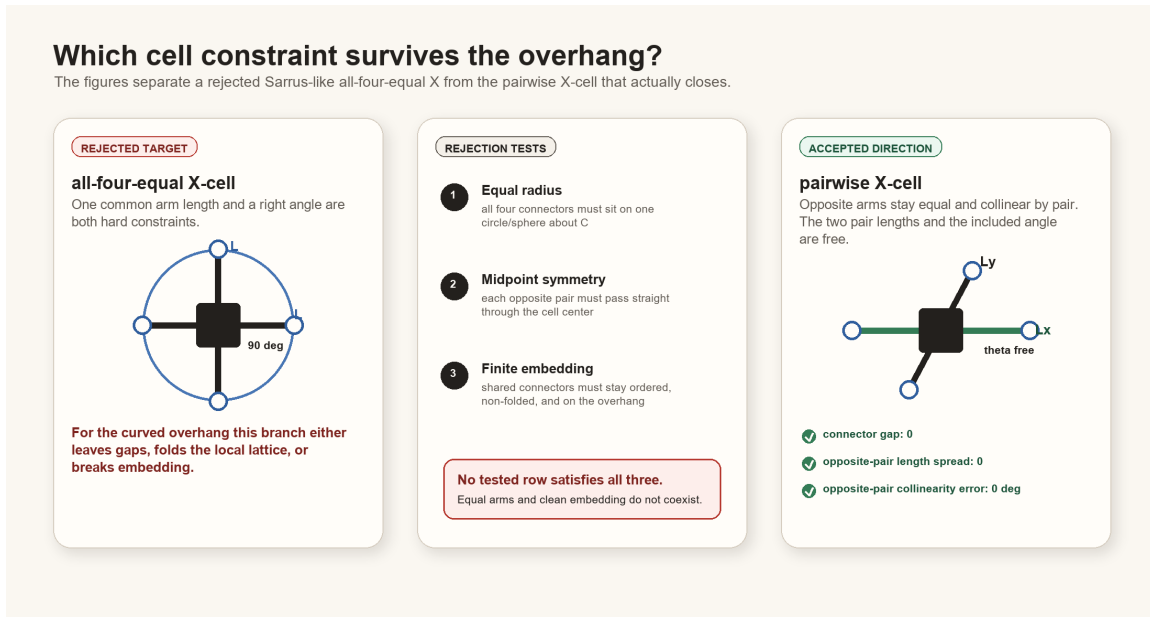


Figure 1. Constraint contract summary. The relevant result is not visual polish; it is which constraints are active, which were rejected, and which residuals must stay visible.

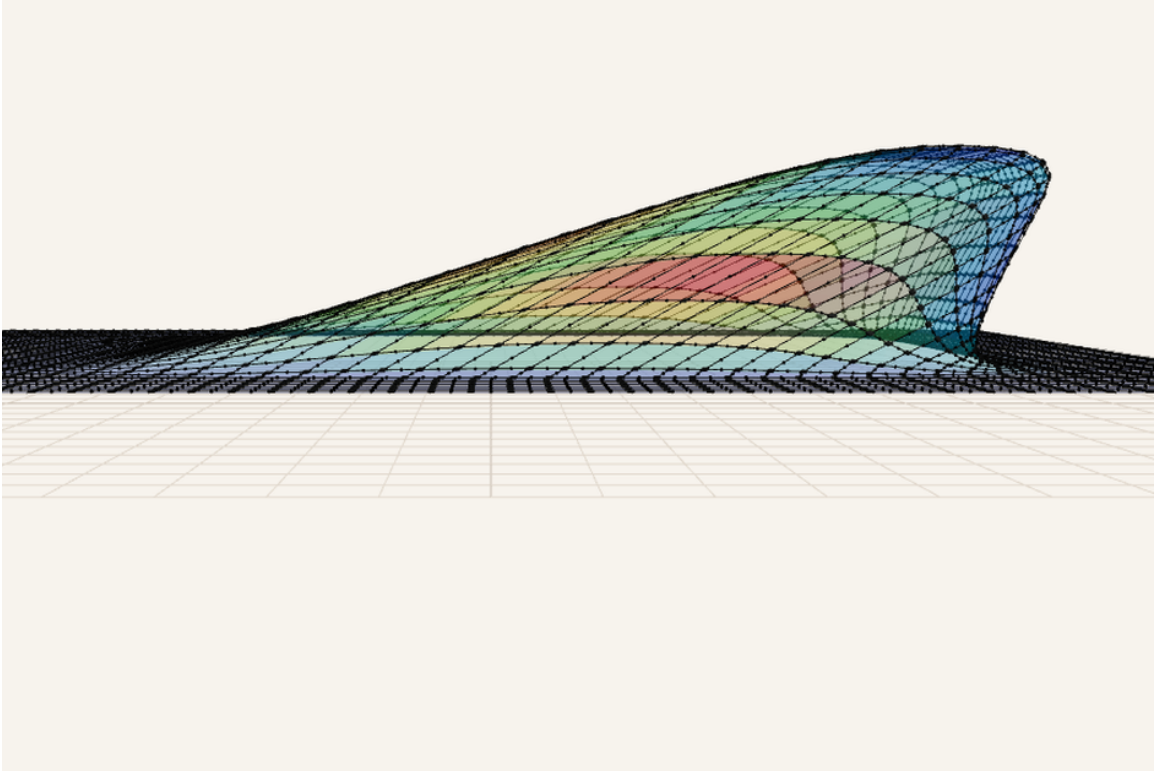


Figure 2. Accepted pairwise X-cell grammar. Opposite pairs are straight and equal about the center, while all-four equality is not claimed.

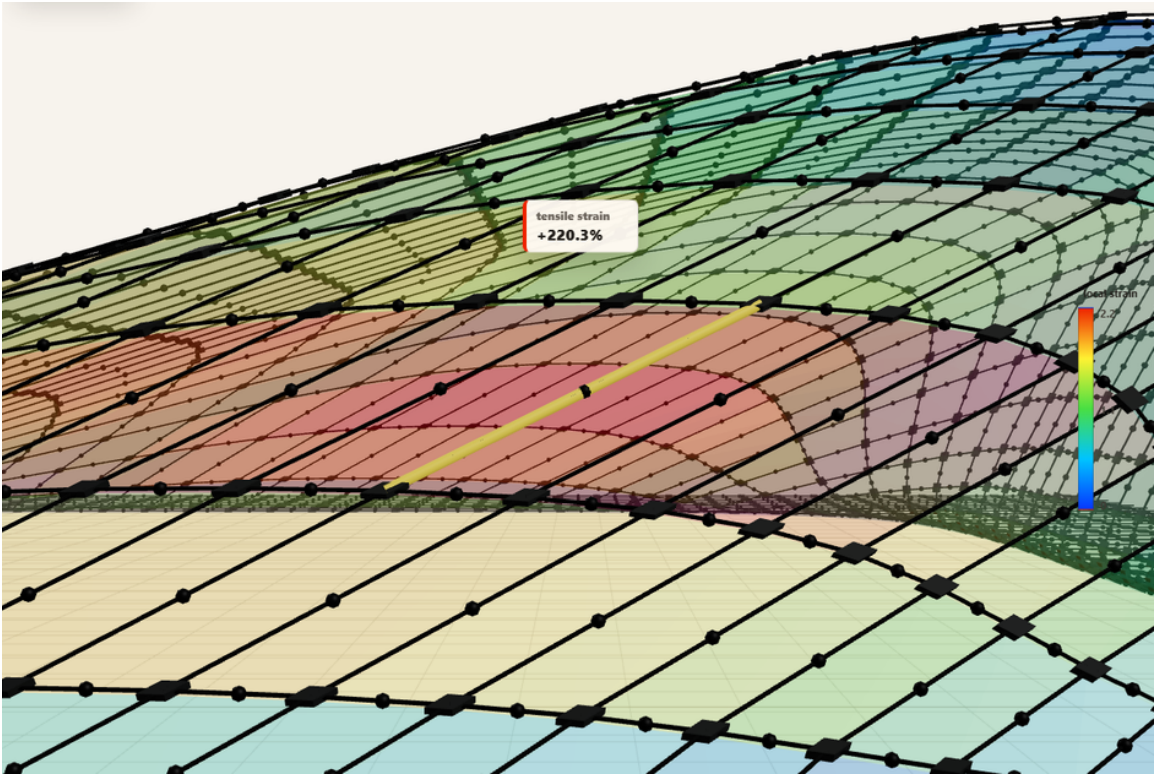


Figure 3. Rejected all-four-equal grammar. This figure belongs in the paper only as history and warning: it shows the kind of constraint claim the current overhang target cannot support.

9 Relationship to Sarrus Motion

The double-layer Sarrus idea is useful because it points toward a mechanism family in which paired links guide out-of-plane motion. The current WaveVis X-cell grammar is not yet a full physical Sarrus unit with fabricated hinge axes, layer spacing, hard stops, material thickness, and actuator geometry. A future physical Sarrus cell would need:

1. two rigid layers or two hinge planes;
2. explicit hinge axes;
3. bar lengths and offsets;
4. collision/clearance checks;
5. a state variable connecting actuation to layer separation or shear;
6. manufacturing tolerances;
7. physical test data.

The present model is a constraint-screening step before that build. It says which geometric claims are honest enough to carry forward.

10 Validation Plan

Before a candidate can be called a mechanism candidate, these checks must pass:

- interior cells have four legs;
- each opposite pair remains collinear and equal within the accepted tolerance;
- each adjacent connector has exactly two endpoint uses;
- no physical connector point has more than two endpoint uses;
- endpoint gaps are zero or explicitly reported;
- connector line residual and overshoot are reported when nonzero;
- full mechanism visibility is preserved in side, top, front, isometric, desktop, and mobile views;

- rejected filler bridges, masks, fake couplers, and surface-only renders do not reappear.

11 Discussion

The main design decision is now explicit. If Alan wants the strict all-four-equal X-cell, the current overhang target must be weakened or a different topology must be found. If Alan wants the localized overhang target, the pairwise X-cell branch is the honest current branch because it preserves visible cells, straight opposite pairs, and two-cell connectors while dropping the unsupported all-four equality claim. That is a tradeoff, not a bug.

This paper should prevent the earlier failure where a smooth render looked finished while the actual mechanism constraints had changed behind the scenes. Future work should make that tradeoff visible in the UI and in the paper instead of making Alan rediscover it through screenshots.

12 Materials and Methods

The mechanism is built from the current lattice centers. The node centers are optionally smoothed by a five-pass local kernel with boundary centers locked. Each pass uses a center weight of 0.46, direct-neighbor weights of 0.09, and diagonal-neighbor weights of 0.045. The smoothing is a display/mechanism readability operation, not a physical relaxation.

The connector solver is deterministic for each row and column chain. It creates adjacent connectors, writes endpoints onto the two neighboring cell frames, records connector uses, and then computes residual metrics. The metrics are used as gates and as paper evidence.

13 Data availability

The proof figures are stored in `proofs/figures/`. The old infeasibility record is preserved in the existing connector-contact proof PDF and source. No physical build data is included.

14 Code availability

The active code is in `src/xCellMechanism.ts`, `src/latticeGeometry.ts`, and `src/LatticeViewer3D.tsx`.

15 Additional information

144

This paper is standalone. It covers only the double-layer Sarrus / pairwise X-cell mechanism branch, the rejected all-four-equal claim, connector occupancy, and the residual checks that keep the visible mechanism honest.

145

146

147

References

148

1. K. H. Hunt, *Kinematic Geometry of Mechanisms* (Oxford University Press, 1978). 149
2. J. M. McCarthy, *Geometric Design of Linkages* (Springer, 2000). 150
3. K. J. Waldron, Overconstrained Linkage Geometry by Solution of Closure Equations. Part I. Method of Study. *Mechanism and Machine Theory* **8**, 95–104 (1973). 151
152
4. M. Schenk, S. D. Guest, Geometry of Miura-Folded Metamaterials. *Proceedings of the National Academy of Sciences* **110**, 3276–3281 (2013). 153
154
5. J. L. Silverberg *et al.*, Using Origami Design Principles to Fold Reprogrammable Mechanical Metamaterials. *Science* **345**, 647–650 (2014). 155
156
6. E. T. Filipov, T. Tachi, G. H. Paulino, Origami Tubes Assembled into Stiff, Yet Reconfigurable Structures and Metamaterials. *Proceedings of the National Academy of Sciences* **112**, 12321–12326 (2015). 157
158
159
7. C. Coulais, E. Teomy, K. de Reus, Y. Shokef, M. van Hecke, Combinatorial Design of Textured Mechanical Metamaterials. *Nature* **535**, 529–532 (2016). 160
161
8. M. Kadic, G. W. Milton, M. van Hecke, M. Wegener, 3D Metamaterials. *Nature Reviews Physics* **1**, 198–210 (2019). 162
163